3

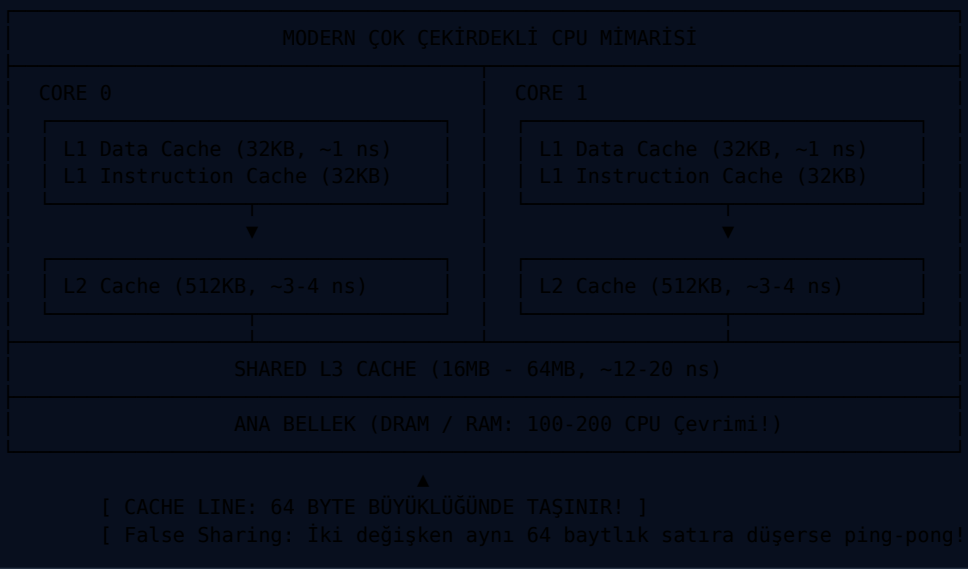
BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Donanım Mimarisi: CPU Önbellekleri (L1/L2/L3), Store Buffer & False Sharing

Hardware Reality: CPU Caches (L1/L2/L3), Store Buffers & False Sharing

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

CPU o kadar hızlı koşar ki, ana belleğe (RAM) gitmek onun için 3 ay süren bir gemi yolculuğu gibidir! Bu yüzden masasının hemen üstüne minicik çekmeceler koymuştur: L1 (kol mesafesinde), L2 (odanın köşesinde), L3 (koridorda). CPU veriyi belleğe giderken tek bir bayt olarak değil, her defasında 64 baytlık paket (Cache Line) halinde getirir. Eğer iki farklı işçi, aynı 64 baytlık paketin içindeki farklı iki kutuyu güncellemeye çalışırsa, CPU'lar sürekli birbirinin çekmecesini geçersiz kılar (False Sharing). İşçiler birbirine dokunmasa bile sistem yerlerde sürünür!

🇬🇧 ENGLISH (ELI5)

CPUs execute so fast that fetching from RAM feels like waiting 3 months for cargo delivery! To survive, the core keeps micro-drawers on its desk: L1 (arm's length, 1ns), L2 (across the room, 4ns), L3 (down the hall, 15ns). Hardware never transfers a lone 4-byte integer from RAM; it fetches an entire 64-byte Cache Line. If Thread A on Core 0 modifies variable X, and Thread B on Core 1 modifies variable Y, but X and Y sit inside the same 64-byte line, cache coherency protocols (MESI) invalidate both cores' caches repeatedly (False Sharing)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MESI Protokolü ve False Sharing derinliği: 1) Cache Line: Modern x86 ve ARM64 mimarilerinde önbellek aktarım birimi 64 bayttır. 2) MESI Protokolü (Modified, Exclusive, Shared, Invalid): Bir çekirdek bir cache line'a yazdığında, donanım veri yoluna 'Invalidate' sinyali gönderir. Diğer tüm çekirdeklerin o satırdaki L1/L2 kopyaları anında çöpe gider ve RAM/L3'ten tekrar çekilmek zorunda kalır. 3) False Sharing Felaketi: Mantıksal olarak bağımsız iki `volatile` veya atomik değişken bellekte bitişik tanımlanmışsa (ör. `long a; long b;`), farklı thread'ler bunlara bağımsız yazsa bile aynı 64 baytlık satır çekirdekler arasında pinpon topu gibi gidip gelir (Bus saturation). 4) Çözüm: Java 8+ ile gelen `\@jdk.internal.vm.annotation.Contended` anotasyonu veya manuel alan doldurma (Padding: 128 bayt aralık bırakma).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Cache Coherency & False Sharing mechanics: 1) The 64-Byte Unit: Memory subsystems always populate L1/L2 caches in contiguous 64-byte aligned blocks. 2) MESI Protocol State Flips: When Core 0 writes to an address, it transitions the cache line from Shared to Modified, forcing Core 1 to mark its local copy as Invalid. 3) False Sharing Inefficiency: If Thread 1 frequently writes to `val1` and Thread 2 writes to `val2`, and both fields inhabit the same 64-byte cache line, the CPU interconnect is choked with cache-line invalidation traffic despite zero algorithmic overlap. 4) Remediation: JVM `\@Contended` annotation (requires `-XX:-RestrictContended`) pads instances with 128 dummy bytes, ensuring isolated cache lines.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: False Sharing Korumalı Cache-Line Padded Sayaç
public class CachePaddedCounter {
    // 64-baytlık satırı doldurmak için boşluk (Padding) bırakma tekniği
    public static class PaddedAtomicLong {
        // Nesne başlığı (Object Header) ~12-16 byte
        // Önceki 56 bayt doldurma
        public volatile long p1, p2, p3, p4, p5, p6, p7;

        // Asıl değerimiz (8 bayt)
        public volatile long value = 0L;

        // Sonraki 56 bayt doldurma
        public volatile long p8, p9, p10, p11, p12, p13, p14;
    }

    void main() {
        var counter = new PaddedAtomicLong();
        counter.value = 42L;
        IO.println("Cache-line ayrılmış güvenli sayaç değeri: " + counter.value);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'Dizideki farklı indekslere farklı thread'ler yazarsa race condition olur mu?' diye sorar. Yanıt: 'Mantıksal race condition olmaz ama dizinin bitişik elemanları aynı 64-baytlık cache line'a düşeceği için donanımsal False Sharing felaketi yaşanır ve performans 50 kat düşebilir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

High-Impact Interview Trap: 'If Thread A writes to array[0] and Thread B writes to array[1], is there any concurrency penalty?' Senior Answer: 'There is no semantic data race, but there is severe False Sharing. Because array[0] and array[1] share a single 64-byte cache line, CPU cores continually invalidate each other's L1 caches, slashing throughput by up to 50x!'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• CPU veriyi RAM'den 64 baytlık Cache Line blokları halinde transfer eder.

• False Sharing, bağımsız değişkenlerin aynı cache satırını paylaşmasıyla oluşur.

• Yüksek performanslı kuyruklar (Disruptor) ve sayaçlar (LongAdder) cache padding kullanır.

🎯 Key Takeaways for the Interview

• CPUs fetch memory in atomic 64-byte cache lines, not single primitive bytes.

• False Sharing occurs when unrelated variables occupy the same 64-byte cache line.

• High-throughput engines (e.g. LMAX Disruptor) pad memory to isolate cache lines.